JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.temporal

Class JulianFields

java.lang.Object
    java.time.temporal.JulianFields

```
public final class JulianFields
extends Object
```

A set of date fields that provide access to Julian Days.

The Julian Day is a standard way of expressing date and time commonly used in the scientific community. It is expressed as a decimal number of whole days where days start at midday. This class represents variations on Julian Days that count whole days from midnight.

The fields are implemented relative to EPOCH_DAY. The fields are supported, and can be queried and set if EPOCH_DAY is available. The fields work with all chronologies.

Implementation Requirements:

This is an immutable and thread-safe class.

Since:

1.8

Field Summary

Fields

Modifier and Type	Field	Description
static TemporalField	JULIAN_DAY	Julian Day field.

static TemporalField	MODIFIED_JULIAN_DAY	Modified Julian Day field.
static TemporalField	RATA_DIE	Rata Die field.

Method Summary

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

JULIAN_DAY

```
public static final TemporalField JULIAN_DAY
```

Julian Day field.

This is an integer-based version of the Julian Day Number. Julian Day is a well-known system that represents the count of whole days since day 0, which is defined to be January 1, 4713 BCE in the Julian calendar, and -4713-11-24 Gregorian. The field has "JulianDay" as 'name', and 'DAYS' as 'baseUnit'. The field always refers to the local date-time, ignoring the offset or zone.

For date-times, 'JULIAN_DAY.getFrom()' assumes the same value from midnight until just before the next midnight. When 'JULIAN_DAY.adjustInto()' is applied to a date-time, the time of day portion remains unaltered. 'JULIAN_DAY.adjustInto()' and 'JULIAN_DAY.getFrom()' only apply to `Temporal` objects that can be converted into `ChronoField.EPOCH_DAY`. An `UnsupportedTemporalTypeException` is thrown for any other type of object.

In the resolving phase of parsing, a date can be created from a Julian Day field. In strict mode and smart mode the Julian Day value is validated against the range of valid values. In lenient mode no validation occurs.

Astronomical and Scientific Notes

The standard astronomical definition uses a fraction to indicate the time-of-day, thus 3.25 would represent the time 18:00, since days start at midday. This implementation uses an integer and days starting at midnight. The integer value for the Julian Day Number is the astronomical Julian Day value at midday of the date in question. This amounts to the astronomical Julian Day, rounded to an integer `JDN = floor(JD + 0.5)`.

ISO date	Julian Day Number	Astronomical Julian Day
1970-01-01T00:00	2,440,588	2,440,587.5
1970-01-01T06:00	2,440,588	2,440,587.75
1970-01-01T12:00	2,440,588	2,440,588.0
1970-01-01T18:00	2,440,588	2,440,588.25
1970-01-02T00:00	2,440,589	2,440,588.5
1970-01-02T06:00	2,440,589	2,440,588.75
1970-01-02T12:00	2,440,589	2,440,589.0

Julian Days are sometimes taken to imply Universal Time or UTC, but this implementation always uses the Julian Day number for the local date, regardless of the offset or time-zone.

MODIFIED_JULIAN_DAY

```
public static final TemporalField MODIFIED_JULIAN_DAY
```

Modified Julian Day field.

This is an integer-based version of the Modified Julian Day Number. Modified Julian Day (MJD) is a well-known system that counts days continuously. It is defined relative to astronomical Julian Day as `MJD = JD - 2400000.5`. Each Modified Julian Day runs from midnight to midnight. The field always refers to the local date-time, ignoring the offset or zone.

For date-times, 'MODIFIED_JULIAN_DAY.getFrom()' assumes the same value from midnight until just before the next midnight. When 'MODIFIED_JULIAN_DAY.adjustInto()' is applied to a date-time, the time of day portion remains unaltered. 'MODIFIED_JULIAN_DAY.adjustInto()' and 'MODIFIED_JULIAN_DAY.getFrom()' only apply to `Temporal` objects that can be converted into `ChronoField.EPOCH_DAY`. An `UnsupportedTemporalTypeException` is thrown for any other type of object.

This implementation is an integer version of MJD with the decimal part rounded to floor.

In the resolving phase of parsing, a date can be created from a Modified Julian Day field. In strict mode and smart mode the Modified Julian Day value is validated against the range of valid values. In lenient mode no validation occurs.

Astronomical and Scientific Notes

ISO date	Modified Julian Day	Decimal MJD
1970-01-01T00:00	40,587	40,587.0
1970-01-01T06:00	40,587	40,587.25
1970-01-01T12:00	40,587	40,587.5
1970-01-01T18:00	40,587	40,587.75

```
| 1970-01-02T00:00  |           40,588  |       40,588.0   |
| 1970-01-02T06:00  |           40,588  |       40,588.25  |
| 1970-01-02T12:00  |           40,588  |       40,588.5   |
```

Modified Julian Days are sometimes taken to imply Universal Time or UTC, but this implementation always uses the Modified Julian Day for the local date, regardless of the offset or time-zone.

RATA_DIE

```
public static final TemporalField RATA_DIE
```

Rata Die field.

Rata Die counts whole days continuously starting day 1 at midnight at the beginning of 0001-01-01 (ISO). The field always refers to the local date-time, ignoring the offset or zone.

For date-times, 'RATA_DIE.getFrom()' assumes the same value from midnight until just before the next midnight. When 'RATA_DIE.adjustInto()' is applied to a date-time, the time of day portion remains unaltered. 'RATA_DIE.adjustInto()' and 'RATA_DIE.getFrom()' only apply to `Temporal` objects that can be converted into `ChronoField.EPOCH_DAY`. An `UnsupportedTemporalTypeException` is thrown for any other type of object.

In the resolving phase of parsing, a date can be created from a Rata Die field. In strict mode and smart mode the Rata Die value is validated against the range of valid values. In lenient mode no validation occurs.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.